Race-proof Team Formation

Design notes. Every hard guarantee is enforced by Postgres in a single transaction, and proven by a test that fails if the guarantee is removed.



1. Data model

`teams` holds the denormalised `member_count`, `max_size` and a `version` that bumps on each change. Three constraints carry correctness:

- `memberships` primary key `(hackathon_id, user_id)`: one team per person. A composite foreign key to `teams (id, hackathon_id)` stops a wrong hackathon id from getting around it.
- Partial unique index `one_leader_per_team`: never two leaders.
- `CHECK (member_count BETWEEN 0 AND max_size)`: a last line of defence.

The API connects as a role that can only `SELECT` and `INSERT` on `audit_events`, so the log is append-only by database grant.

2. Preventing races

A seat is taken by one statement: `UPDATE teams SET member_count = member_count + 1 ... WHERE id = $1 AND member_count < max_size`. Racers block on the row lock. When the winner commits, `READ COMMITTED` re-evaluates the `WHERE` against the new row and updates 0 rows, so there is no gap between check and act. Racing into two teams fails on the membership key and rolls back that seat.

- **Lock order:** team, then memberships, then the user row, then requests and codes. Each transaction locks at most one team. The user row serialises joining one team against asking to join another, so no request is left pending. Two tests force the dangerous interleavings with `pg_blocking_pids`, because they almost never occur by chance.
- **Authorisation after the lock, in a new statement.** A subquery inside the lock-taking `UPDATE` keeps its old snapshot on re-check, so a demoted leader would pass.
- **Why not `SERIALIZABLE`:** correct, but a stampede mostly aborts and retries. **Why not Redis:** a second source of truth that fixes nothing.

The tests can fail: `race.mutation-check` swaps in a read-then-write seat and asserts several winners. The Race Arena shows the same bug live.

3. Idempotency

The key row commits in the same transaction as the change. A duplicate blocks on the primary key, then replays the stored answer. Domain failures roll back to a savepoint and keep the claim, so `TEAM_FULL` replays as `TEAM_FULL`. A crash rolls back both, so no "in progress" state can get stuck. This is only valid because every side effect is local to one ACID database. Invite creation opts out, since storing its response would store the plaintext code.

4. Real-time

The audit row committed with a change is the outbox, so there is no dual write. `pg_notify` runs inside the transaction and is delivered only on commit. Each replica holds one direct connection for `LISTEN`, reads the committed rows and pushes them to its own SSE clients.

Ids from `bigserial` are assigned at insert, not commit, so `Last-Event-ID` alone can skip an event. Clients therefore use **snapshot plus versioned deltas**: apply an event only if its version is newer, and refetch on every reconnect. A miss means briefly stale, never wrong.

5. Trade-offs

- **Denormalised count:** fast board reads, with drift risk covered by one transaction and an invariant checker run after tests, seeding and chaos.
- **SSE over WebSocket:** updates flow one way, and reconnect comes free.
- **Postgres rate limits over Redis:** durable and shared with no new infrastructure. Attempts reserve first and refund on success, so bursts cannot slip through. Cost: a write per attempt.
- **`NOTIFY` in the transaction:** cannot be lost, but takes a global lock at commit. Fine here; not at very large scale.
- **Same origin:** Fastify serves the SPA, so cookies, API and SSE share one origin. No third-party cookie problem.
- **Free tier:** 0.1 CPU limits live latency, so Compose results are reported beside the live ones.

6. Results

Local Compose stack, 12 logical CPUs, seeded 10k users and 2k teams, k6 on the same machine (10 Nov 2026):

- **Board reads, 100 VUs:** p95 114.1 ms against a 200 ms budget, 1,811 requests/s, 0 failures of 326,379.
- **With 10 writers and 300 open SSE streams:** reader p95 107.2 ms, 0 failures in any scenario.
- **Chaos:** 8 replicas killed under a write storm with 1,280 seats taken: all invariants hold, 0 intents abandoned.
- **Tests:** 122 API tests on real Postgres and 9 browser tests, including keyboard-only and axe. Live at lastseat-b4mi.onrender.com.

7. One more week

- Commit-ordered event log through logical replication.
- Waitlist with automatic seat offers and a timed hold.
- WebAuthn passkeys.
- An organiser UI, and partitioning the audit table per hackathon.
- k6 at 1,000 VUs on paid compute.